

ReactEdge whitepaper	1
1. The Problem: Complexity Creates Specialist Dependency	1
2. The ReactEdge Approach: Reduce the surface a developer must understand	1
2.1. What ReactEdge Abstracts	2
2.2. Capability Architecture: Shared Plumbing, Independent Business Behaviour	3
2.3. Contracts as Architectural Boundaries	3
2.4. State Ownership, Signals and Synchronisation	4
3. Orchestration: Performance and Verification Concerns	6
4. SSR, Build and Deployment	7
4.1. SSR service	7
4.2. Deployment pipeline	8
4.3. Independent Deployment	9
4.4. Reverting a ReactEdge Capability	9
5. Progressive Adoption and Reversibility	9
6. Two Reference Capabilities: USP and Product Gallery	9
8. AI-Operable Architectural Boundaries: MCP	11
9. What ReactEdge Does Not Abstract	11
10. Consequences for Engineering Teams	12
11. Conclusion: Making Complex Systems Boring to Change	12

ReactEdge whitepaper

1. The Problem: Complexity Creates Specialist Dependency

Complex commerce frontends accumulate knowledge that is difficult to maintain over time. When each feature's boundaries are blurred, maintaining the codebase for one feature has more than one concern. Shared ownership, testing surface, risk to break critical features are bigger problems.

It remains possible to create shared libraries to reuse some of the code. But it is common to find these environments to become coupled over time. In particular, it is common to see some concerns to be shared (CSS, JavaScript, Google dependencies, SSR, State). The problem is not simply that specialists are expensive. But rather the inability to remain able to evolve the system by parts yields to team friction, risk and loss of velocity.

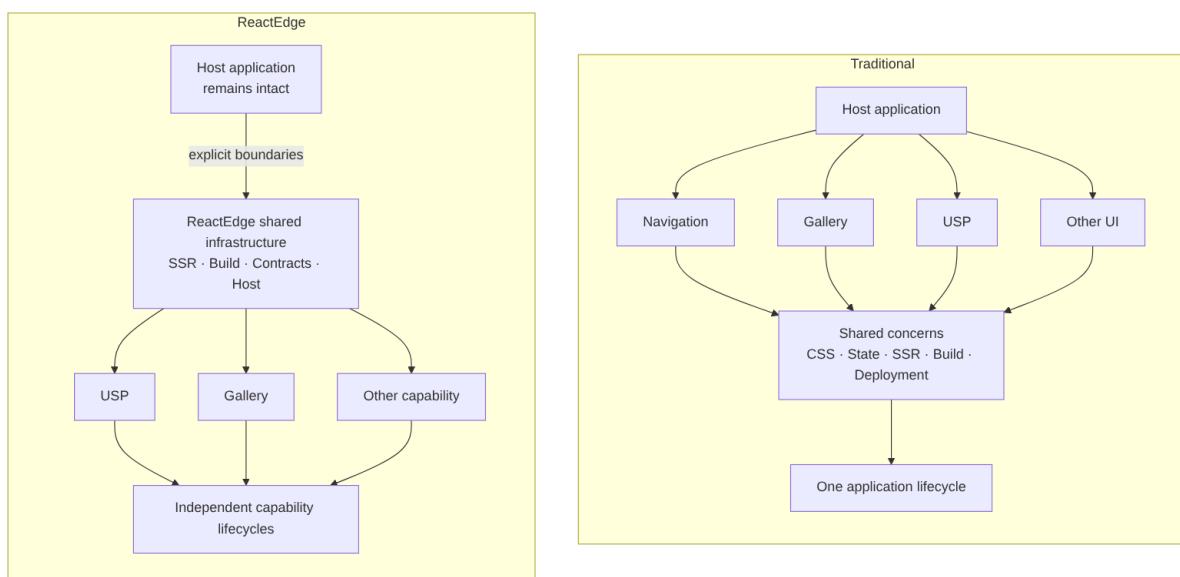
2. The ReactEdge Approach: Reduce the surface a developer must understand

ReactEdge creates UI capabilities in isolated Vite environments that share a common architecture. Each capability follows a predictable structure, including a public API, SSR entrypoints, an NPM package boundary, structured configuration and declared dependencies.

ReactEdge is designed around making these boundaries explicit. The data and external services required by a capability are defined at its boundary rather than being implicitly shared with the rest of the application.

This explicit boundary is what enables genuine isolation. Because the capability's requirements are known and contained, it can be developed and tested locally, deployed independently from the host application, and potentially moved between different host systems without carrying implicit dependencies with it.

The result is that capability development remains outside the core application. The core is changed less frequently and can continue to evolve largely independently from the UI capabilities around it. This reduces the surface affected by capability changes and helps preserve the core's stability, testability and upgrade path over time.



2.1. What ReactEdge Abstracts

A ReactEdge capability should not have to solve the infrastructure required to become part of a larger application.

For example, a Product Gallery developer should be concerned with the gallery itself: its HTML, React components, state, styling and data requirements. They should not have to design a new mechanism for mounting the capability, server-side rendering it, loading its configuration, exposing it as a JavaScript or NPM API, instrumenting it, or packaging it for deployment.

ReactEdge therefore provides these concerns as shared infrastructure. This includes:

1. **Embedding and isolation:** a common mechanism mounts capabilities into either the light DOM or Shadow DOM and exposes predictable JavaScript and NPM interfaces.
2. **Configuration and data boundaries:** shared contract loading and validation establish how configuration enters a capability. Runtime capabilities can additionally access host data through defined integrations such as GraphQL.
3. **Rendering:** common browser and SSR mechanisms allow capabilities to participate in both client-side and server-side rendering without implementing that infrastructure independently.
4. **Styling and translation:** capabilities receive common mechanisms for self-contained CSS, optional use of host styles and translated content.
5. **Build and deployment:** shared TypeScript definitions, build conventions and deployment tooling produce capabilities through the same pipeline.
6. **Observability and verification:** common instrumentation and testing conventions make capabilities observable and independently testable.

These abstractions move platform-specialist concerns out of individual capabilities. A capability can still contain substantial business behaviour, but most day-to-day changes can be expressed through familiar frontend primitives such as React, HTML, CSS, state, events and data.

The objective is not to hide complexity. It is to implement recurring infrastructure once, while keeping capability-specific complexity local to the capability that owns it.

2.2. Capability Architecture: Shared Plumbing, Independent Business Behaviour

Each capability owns its public entry point, contract, components, behaviour, styles and tests, while those files consume shared `@reactedge/*` packages for host integration, public APIs and common bootstrap mechanics.

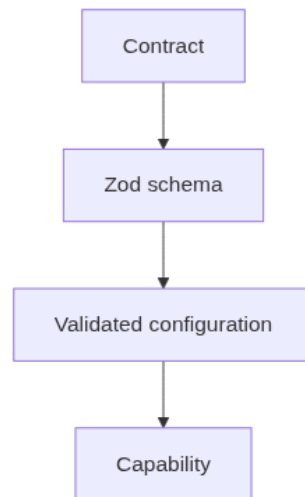
2.3. Contracts as Architectural Boundaries

The contract is the primary mechanism used by ReactEdge to control what configuration enters a capability.

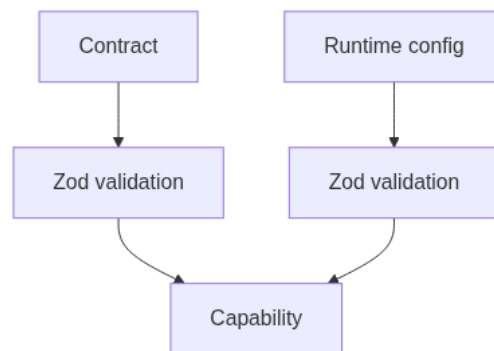
Each capability defines a Zod schema describing the data it accepts. When a contract is loaded, its content is validated against that schema before it is made available to the React application. Data that does not conform to the declared structure is rejected at the boundary.

TypeScript and Zod have complementary roles. TypeScript provides compile-time types for developers working inside the capability, while Zod validates the data received at runtime.

For a Standard capability, the contract is sufficient to describe its configuration:



Runtime capabilities extend this mechanism with a second runtime configuration. This describes contextual data and services that must be supplied by the host environment:

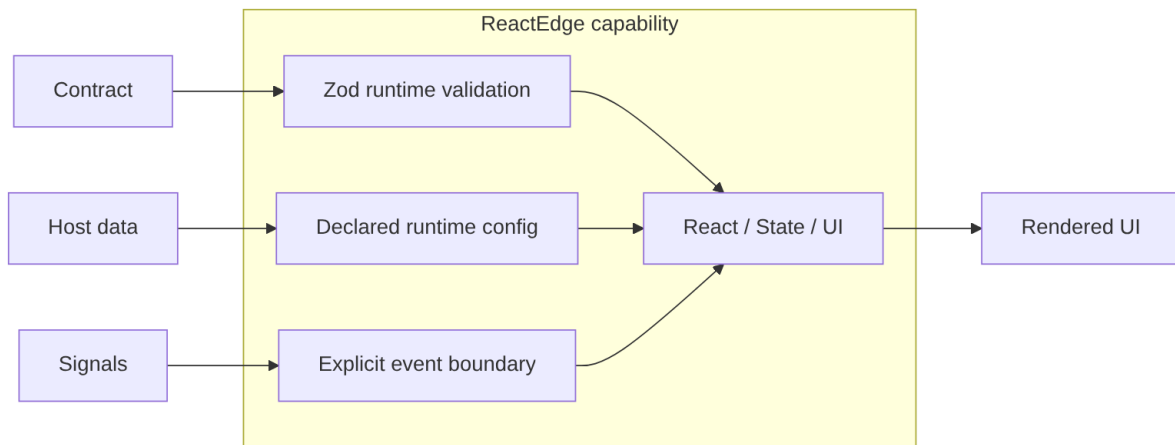


This distinction allows a capability to declare not only its configurable behaviour, but also the host-specific information required for it to operate.

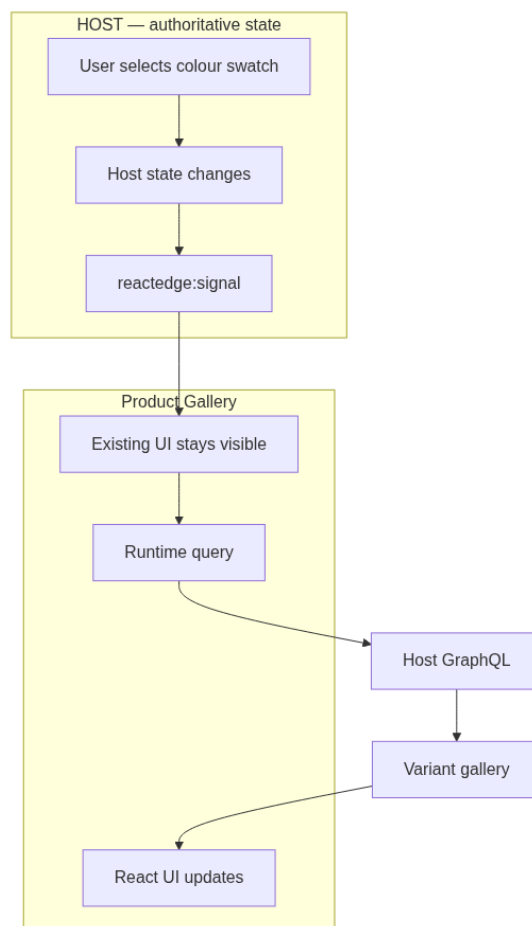
2.4. State Ownership, Signals and Synchronisation

The host owns the authoritative application state. ReactEdge capabilities synchronize with that state through explicit signals and runtime data acquisition.

And importantly, asynchronous ReactEdge updates do not have to destroy the currently valid UI while waiting for new data.



ReactEdge uses explicit signals to communicate selected host state changes across the capability boundary. In the current reference implementation, the signal protocol is implemented by the host adapter and the consuming capability. The protocol is not yet represented as a standalone declarative contract, making this boundary less machine-discoverable than configuration and runtime-data boundaries.



3. Orchestration: Performance and Verification Concerns

Most of the ReactEdge architecture described so far is concerned with reducing engineering friction, lowering the cost of maintaining UI capabilities and creating predictable boundaries.

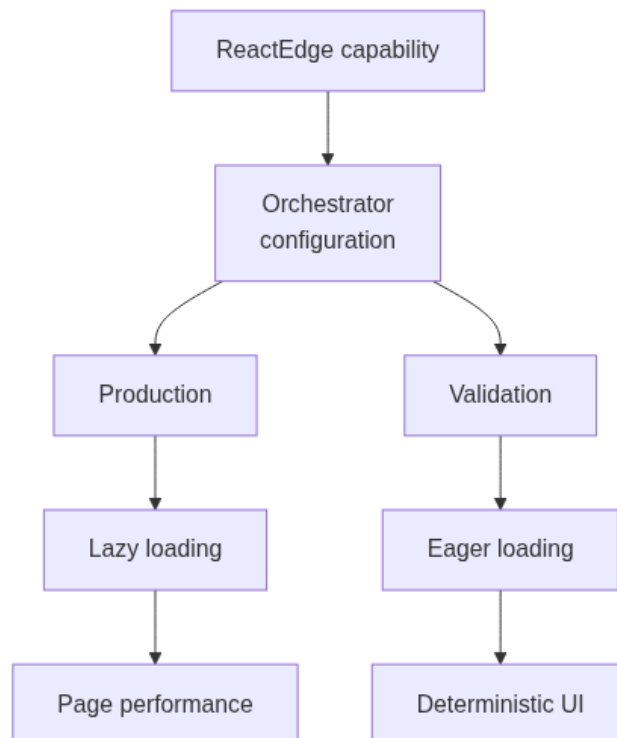
ReactEdge also addresses concerns that emerge when those capabilities run in production.

Capabilities can be configured to lazy load so that expensive UI does not unnecessarily contribute to the critical page load. This allows the host page to become interactive without waiting for every ReactEdge capability to initialise, while more expensive capabilities can load when required. This is particularly important when optimising frontend performance and Lighthouse scores.

Performance optimisation, however, introduces a different problem: runtime behaviour becomes less deterministic. Lazy loading, asynchronous rendering and waiting conditions are common sources of instability in end-to-end tests. A capability may be correct while a test fails simply because the expected UI has not yet been loaded or rendered.

ReactEdge therefore allows the orchestrator to change this behaviour according to the execution context. In production, capabilities can retain their configured lazy-loading strategy. During validation and E2E testing, lazy loading can be disabled so that capabilities are loaded deterministically.

The same capability can therefore be operated differently according to the concern being optimised:



4. SSR, Build and Deployment

4.1. SSR service

ReactEdge capabilities support three SSR strategies: **Static**, **Dynamic** and **Disabled**. The strategy determines whether server-rendered HTML is produced and when that rendering occurs.

Static SSR is used when the data required to render a capability is known at deployment time. ReactEdge generates the server-rendered HTML during the build and includes it in the deployed artefacts. USP is an example of this strategy. No runtime rendering request is required, allowing the capability to behave similarly to native host UI without depending on an additional runtime rendering or caching layer.

Dynamic SSR is used when the data required to render a capability must be resolved at request time. MegaMenu and Product Gallery are examples. ReactEdge provides a shared SSR API that resolves the required runtime data and produces the server-rendered view. Because data acquisition and rendering happen at runtime, caching becomes more important. GraphQL responses can, for example, be cached through Cloudflare Workers to reduce repeated data resolution.

Disabled SSR is used when server-side rendering is not required. The capability is rendered entirely on the client and does not produce server-rendered HTML.

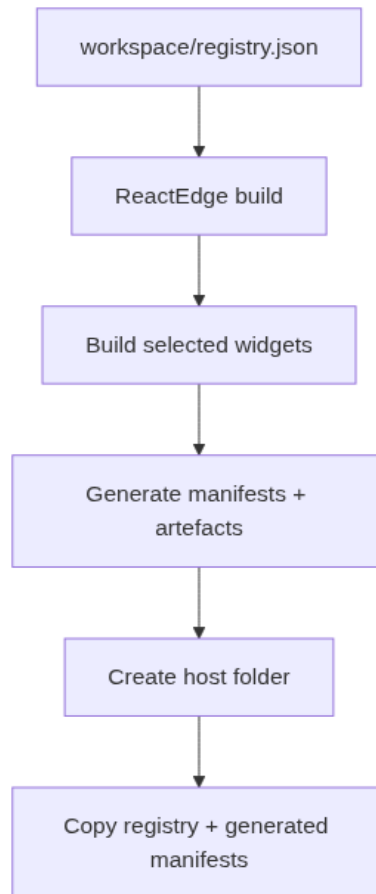
The SSR strategy is therefore part of the capability's deployment configuration rather than something each capability implements independently. ReactEdge provides the rendering infrastructure, while each deployment selects the strategy appropriate to the capability and its data requirements.

4.2. Deployment pipeline

The ReactEdge deployment pipeline is driven by a workspace registry. The registry describes the capabilities to deploy and the configuration associated with each deployment.

During the build, ReactEdge reads the registry and builds the corresponding Vite environments. The resulting artefacts, which can include JavaScript, CSS, contract data and pre-rendered HTML when Static SSR is enabled, are assembled into capability manifests. ReactEdge then creates the host deployment folder containing a copy of the registry, the generated manifests and the capability artefacts.

The registry therefore acts as the link between capability source code and a particular deployment. The capability defines what can be built, while the registry defines which capabilities and configurations are deployed to a host.



4.3. Independent Deployment

Once a capability has been built and its artefacts deployed, the host application does not need to be rebuilt or redeployed.

For the Magento integration, the ReactEdge cache is invalidated so that the updated capability artefacts are used:

```
bin/magento cache:clean reactedge
```

4.4. Reverting a ReactEdge Capability

ReactEdge capabilities can be disabled without removing the native host implementation they replace. This provides a direct path back to the original UI without requiring a code deployment.

For example, the Product Gallery can be disabled through Magento configuration:

```
bin/magento config:set reactedge/productgallery/enabled 0
```

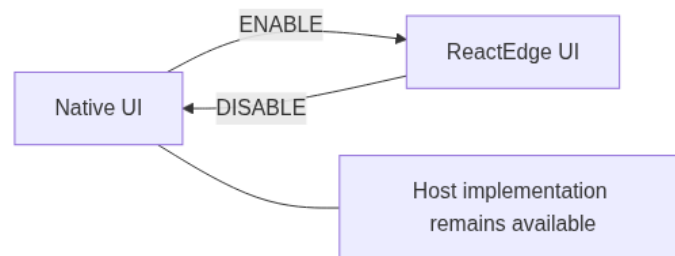
```
bin/magento cache:flush
```


The ReactEdge integration then stops replacing the native Product Gallery, allowing the host implementation to become active again.

5. Progressive Adoption and Reversibility

ReactEdge capabilities are designed for progressive adoption. Introducing a capability does not require deleting the host implementation it replaces. The host capability can remain available as the fallback path.

In the Magento adapter, enabling a ReactEdge capability suppresses the corresponding native frontend block and mounts its ReactEdge replacement. Disabling it restores the native capability.



6. Two Reference Capabilities: USP and Product Gallery

Two reference capabilities are used to exercise the ReactEdge architecture at different levels of complexity.

USP: minimum capability complexity

USP is intentionally simple and serves a role similar to a Hello World implementation. Despite its limited business logic, it exercises the complete ReactEdge capability lifecycle, including contract loading and validation, DOM mounting, styling, observability, build, deployment and automated testing.

Its simplicity makes it useful for understanding and validating the architecture without introducing significant business complexity.

Product Gallery: real commerce complexity

Product Gallery uses the same capability architecture but introduces concerns found in a real commerce UI. These include Magento data acquisition through GraphQL, configurable product selection, host signals, asynchronous data updates and richer user interaction.

The value of these two reference capabilities is in the contrast. USP demonstrates that the architecture remains lightweight for a simple UI, while Product Gallery tests whether the

same boundaries and shared infrastructure continue to work when the capability becomes substantially more complex.

7. Architectural Conformance

Isolation does not mean that every part of a ReactEdge capability is unconstrained. Capabilities share structural conventions so that the platform can build, validate and operate them consistently.

Capability files can broadly be divided into three categories:

1. **Required and invariant files.** These provide common bootstrap and integration behaviour. They are expected to remain aligned with the ReactEdge reference implementation and are not intended to contain capability-specific behaviour.
2. **Required capability files.** These files must exist because they form part of the capability architecture, but their implementation is specific to the capability. Contracts and their schemas are examples of this category.
3. **Capability-owned files.** These contain the implementation that teams are expected to evolve, such as React components, hooks, services, state and styling.

This distinction allows ReactEdge to preserve a predictable architecture without prescribing the business implementation of each capability.

Architectural conformance can be checked automatically. ReactEdge's MCP tooling can inspect a capability and verify that required files exist and that invariant files still match the expected platform structure.

8. AI-Operable Architectural Boundaries: MCP

ReactEdge's capability design is primarily intended to make features easier for developers to understand and modify independently, without requiring in-depth knowledge of the host platform. Because capabilities follow a predictable architecture and expose explicit boundaries, those same boundaries can also be made available to AI agents through MCP.

MCP exposes information already defined by the ReactEdge architecture rather than creating a separate representation of the system. An agent can discover the capabilities configured in the registry, inspect their contracts and integration requirements, validate configuration against a capability schema, and verify that a capability continues to conform to the expected structure.

The current MCP surface includes:

- `list_widgets` to discover configured capabilities
- MCP resources to inspect capability information
- `validate_contract` to validate contract data against its schema
- `validate_structure` to verify architectural conformance

MCP does not replace the existing engineering pipeline. Builds, tests and commands such as `npm run verify` remain authoritative. MCP provides a machine-operable interface to the architectural boundaries that already exist.

This follows the same principle used throughout ReactEdge: rather than requiring increasingly capable agents to understand an increasingly large system, reduce the technical surface they need to understand in order to make a safe change.

9. What ReactEdge Does Not Abstract

ReactEdge does not introduce an additional extension mechanism around each capability. Doing so would add another layer of inheritance, plugins or overrides that developers would need to understand before modifying the UI.

Instead, the system owner receives a small capability codebase whose React components, behaviour and styling can be modified directly. This keeps capability-specific changes close to familiar frontend primitives rather than requiring another framework-specific extension model.

There is a trade-off. Because capability code can be modified directly, the system owner is responsible for reconciling local changes when adopting upstream ReactEdge updates. ReactEdge currently accepts this responsibility in exchange for keeping capability codebases small and directly understandable.

This is an intentional constraint of the current architecture. Further separation between vendor-owned and capability-owned code may become useful as ReactEdge evolves, but introducing it before there is a demonstrated need would add complexity to the model.

10. Consequences for Engineering Teams

The architectural choices described in this paper have consequences for how teams can work with frontend capabilities.

By keeping capabilities small, isolated and structurally predictable, ReactEdge can provide:

- a smaller reasoning surface for each capability;
- clearer ownership boundaries between capabilities and the host;
- less duplication of infrastructure concerns across teams;
- more consistent build, testing and verification processes;
- easier onboarding into an individual capability;
- specialist platform knowledge concentrated in shared infrastructure rather than required for every capability change;
- a more constrained environment for AI-assisted engineering.

11. Conclusion: Making Complex Systems Boring to Change

ReactEdge is not trying to make enterprise commerce simple.

It is trying to make changing one bounded part of a complex commerce system boring, predictable and verifiable.

MCP provides an additional test of whether those boundaries are genuinely explicit. If an agent can discover a capability, understand its declared requirements and verify whether a change conforms to its architecture without reverse-engineering the host application, then the same boundaries designed for developers are becoming machine-operable.

The broader objective remains the same: keep the complexity of the host where it belongs, isolate capability-specific change from that complexity, and make each capability small enough to understand, develop, test, deploy and replace independently.